

Algorytm genetyczny na GPU dla N-wymiarowej kostki Rubika

Architektura, sposób działania i implementacja potoku obliczeniowego
OpenGL Compute

Robert Świta
Politechnika Koszalińska
Katedra Systemów Multimedialnych i Sztucznej Inteligencji

Dokument opisuje potok GPU-GA solvera: rozdział renderowania i obliczeń, reprezentację danych, pętlę ewolucyjną, metrykę fitnessu oraz kluczowe decyzje implementacyjne.

1. Wprowadzenie

Solver układa N-wymiarową kostkę Rubika (rozmiar **SIZE** na wymiar, **SIZE^N** kubików) za pomocą algorytmu genetycznego, w którym osobnikiem jest sekwencja ruchów. Ewaluacja populacji jest problemem **wybitnie równoległym**: **POPULATION_COUNT** osobników, każdy wykonuje **GENES_COUNT** ruchów na **CUBIES_COUNT** kubikach, całkowicie niezależnie od pozostałych. Taka charakterystyka idealnie pasuje do GPU.

Dlatego cała ewaluacja, selekcja, krzyżowanie i sortowanie zostały przeniesione na GPU jako **shadery obliczeniowe OpenGL** (compute shaders, GLSL 4.30). CPU pełni rolę sterownika: przygotowuje bufory, wyzwała kolejne przebiegi (dispatch), odczytuje najlepszego osobnika i integruje wynik z nadrzędną pętlą rozwiązywania kostki klasterek po klastrze.

2. Architektura

Kod dzieli się na dwie rozłączne warstwy:

- **Renderowanie per-okno** (TGLContext) — kontekst OpenGL związany z konkretnym oknem (HDC/HRC), rysowanie sceny 3D. Teoretycznie może istnieć wiele okien patrzących na tę samą scenę.
- **Potok GPU-GA** (statyczna klasa Gpu) — programy shaderów, bufory SSBO/UBO oraz pętla algorytmu genetycznego (ExecuteGA). Wszystko **statyczne i współdzielone** — nie należy do żadnego konkretnego okna.

TGLContext.Handle tworzy kontekst GL, ustawia go jako bieżący i wywołuje Gpu.Init() — kompilacja shaderów wymaga aktywnego kontekstu.

2.1 Pliki shaderów

Shadery to zasoby osadzone w assembly. Wspólny nagłówek Setup.glsl.c (dołączany dyrektywą #include podmienianą przez host) zawiera define'y, struktury, deklaracje buforów i funkcje pomocnicze. Programy obliczeniowe:

- Init.glsl.c — inicjalizacja populacji losowymi ruchami,
- EvaluateMicro.glsl.c / EvaluateMacro.glsl.c — ewaluacja (dwie strategie równoległości),
- Sort.glsl.c — sortowanie bitoniczne populacji,
- SelCrossover.glsl.c — selekcja, krzyżowanie i mutacja.

2.2 Konfiguracja przez #define i rekompilacja

Parametry są stałymi kompilacji w Setup.glsl.c, bo określają **rozmiary tablic** w shaderze (np. local_cubies[CUBIES_COUNT]). Wartości N, SIZE i CUBIES_COUNT są **wstrzykiwane z bieżącej kostki** (SetDefineValue) przed kompilacją. Gdy użytkownik zmieni wymiar lub rozmiar, CreateBuffers wykrywa różnicę (CompiledN/CompiledSize) i wyzwała rekompilację — shader zawsze pasuje do kostki.

3. Reprezentacja danych

3.1 Osobnik i ruch

Osobnik (chromosom) to struktura Specimen. Fitness jest przechowywany jako bity float (floatBitsToUint), dzięki czemu sortowanie bitoniczne może porównywać go jako uint (dla nieujemnych wartości porządek wzorca bitowego pokrywa się z porządkiem liczb).

```
struct Specimen {
    uint Fitness;           // floatBitsToUint(fitness)
    uint MovesCount;        // dlugosc najlepszego prefiksu
    uint Moves[GENES_COUNT]; // sekwencja zakodowanych ruchow
};
```

```
// Kod ruchu: angle(2b) | plane(5b) | axis(3b) | slice(reszta)
Move getMove(uint code) {
    uint angle = code & 3u;
    uint plane = (code >> 2u) & 31u;
    uint axis = (code >> 7u) & 7u;
    uint slice = code >> 10u;
    return Move(axis, slice, plane, angle);
}
```

3.2 Spakowana macierz orientacji kubika

Orientacja pojedynczego kubika to macierz **znakowanej permutacji** $N \times N$ (każdy wiersz ma dokładnie jeden niezerowy element ± 1). Pakujemy ją do jednego uint: N wierszy po BITS_PER_ROW bitów, pole wiersza = (znak $\ll \text{BITS_FOR_COL}$) | kolumna. Tożsamość: wiersz r koduje (kolumna= r , znak +), czyli wartość pola = r . Ułożony kubik ma wartość tożsamości, a jego odległość L_1 wynosi 0.

```
#define BITS_FOR_COL (N <= 4 ? 2 : 3) // bity na indeks kolumny
#define BITS_PER_ROW (BITS_FOR_COL + 1) // + 1 bit znaku

// getRow / setRow operują na polu wiersza wewnątrz uint.
// Dla N <= 8: N wierszy * (<=4 bity) <= 32 bity -> mieści się w uint.
```

Dla N w zakresie $[3, 8]$ cała macierz mieści się w 32 bitach, co pozwala trzymać kubik w rejestrze i obracać go wyłącznie operacjami bitowymi.

3.3 Bufory GPU (kontrakt bindingów)

Wszystkie bufory tworzy klasa `Ssbo` w stałej kolejności, więc licznik bindingów daje indeksy 0–6 dokładnie tak, jak deklaruje `Setup.glsl.c`:

Bind	Bufor	Typ	Zawartość
0	Population	SSBO	Populacja rodziców (<code>Specimen[]</code>)
1	NewPopulation	SSBO	Dzieci — wyjście krzyżowania (ping-pong)
2	Cubies	SSBO	Spakowany stan startowy (uint na kubik)
3	FreeMoves	SSBO	Dozwolone kody ruchów klastra
4	SolvedCubies	SSBO	Indeksy już ułożonych kubików
5	ActiveCubies	SSBO	Indeksy kubików aktywnego klastra
6	Planes	UBO (std140)	Pary osi płaszczyzn obrotu (<code>ivec2[]</code>)

Bufory Cubies jest współdzielony (tylko do odczytu): każdy wątek kopiuje stan startowy do prywatnej kopii roboczej i mutuje wyłącznie ją — współdzielony bufor jest wolny od wyścigów. `countSolved` i `countActive` przekazywane są jako uniformy (a nie przez `.length()` bufora, bo puste SSBO daje niepewne `.length()` na różnych sterownikach).

4. Sposób działania — pętla ewolucyjna

Metoda `Gpu.ExecuteGA()` realizuje jeden przebieg algorytmu genetycznego dla bieżącej kostki i zwraca najlepszego znalezionej osobnika:

```
CreateBuffers(kostka, freeMoves)    // upload + ew. rekompilacja
UseProgram(Init); Dispatch          // Population[0] = losowe sekwencje
for (gen = 0; gen < GENERATIONS_COUNT; ++gen) {
    UseProgram(evalProgram); Dispatch // 1. fitness kazdego osobnika
    UseProgram(Sort); Dispatch*      // 2. sortowanie bitoniczne
    read Population[0]               // 3. najlepszy na indeksie 0
    if (fitness < best) { best = ...; // zapamiętaj rekord
        if (best < Score) break; }   // prog akceptacji
    UseProgram(SelCrossover); Dispatch // 4. selekcja + krzyzowanie
    parentSlot ^= 1;                 // 5. ping-pong buforow
}
return best;
```

Bufory populacji działają w trybie **ping-pong**: rodzice są zawsze pod bindingiem 0, dzieci pod bindingiem 1, a po każdej generacji role buforów się zamieniają.

4.1 Wymiary dispatchu: micro vs macro

Ewaluacja ma dwie strategie równoległości, wybierane na podstawie liczby kubików:

- **Micro** (≤ 64 kubiki): **1 wątek = 1 osobnik**. Kostka trzymana w prywatnej tablicy w rejestrach; wątek sekwencyjnie obraca kubiki i sumuje błąd. Dispatch: $\text{ceil}(\text{POP}/64)$ bloków po 64.
- **Macro** (> 64 kubiki): **1 blok (256 wątków) = 1 osobnik**. Kostka w pamięci współdzielonej bloku; wątki równolegle obracają kubiki, a błąd liczony jest **logarytmiczną redukcją równoległą**. Dispatch: POP bloków.

Init i SelCrossover to zawsze 1 wątek na osobnika (bloki po 64).

5. Ewaluacja i metryka fitnessu

5.1 Symulacja ruchu na spakowanej macierzy

`TurnSingleCubie` sprawdza, czy dany kubik leży w warstwie obracanej przez ruch, i jeśli tak — obraca jego macierz. Przynależność do warstwy wyznacza `getStartCoordinate` (współrzędna domowa kubika wzdłuż osi ruchu) skorygowana o znak. Konwencja osi jest zgodna z CPU (oś 0 = najstarsza cyfra indeksu liniowego, jak w `TMatrix.Coords2Index`). Sam obrót to `RotateMatInt` na dwóch wierszach płaszczyzny obrotu (zamiana/negacja pól — jedna formuła na każdy kąt $90^\circ/180^\circ/270^\circ$).

5.2 Odległość L1 od tożsamości

Miarą „pokręcenia” kubika jest odległość L1 (Manhattan) spakowanej macierzy od tożsamości. Znak siedzi na wysokim bicie pola, więc odbicie osi różni się o $2^{\text{BITS_FOR_COL}} (\geq N)$, dominując nad przesunięciem kolumny — dzięki temu różne orientacje nie „zlewają się” do tej samej wartości L1.

```
uint cubieL1(uint M) {
    uint dist = 0u;
    for (uint r = 0u; r < uint(N); r++) {
        uint field = (M >> (r * BITS_PER_ROW)) & ((1u << BITS_PER_ROW) - 1u);
        dist += (field > r) ? (field - r) : (r - field); // |field_r - r|
    }
    return dist; // 0 <=> kubik ułożony
}
```

5.3 Składowe fitnessu

Fitness stanu (po danym prefiksie ruchów) składa się z dwóch części:

- **Błąd aktywnego klastra** — suma po kubikach klastra: $(\text{maxClusterState} + d) / \text{max_fA}$, gdzie $d = \text{cubieL1}$, $\text{maxClusterState} = \text{MAX_CUBIE_L1} \cdot \text{countActive}$, $\text{max_fA} = \text{maxClusterState}$.

(countActive+1).

- **Naruszenia ułożonych** — liczba kubików ze zbioru SolvedCubies, które przestały być ułożone (każdy dodaje 1).

$MAX_CUBIE_L1 = N \cdot (2^{BITS_FOR_COL} + N - 1)$ to **ściśle górne ograniczenie** L1 pojedynczego kubika — przewyższa osiągalne maksimum, więc całkowity błąd aktywnego klastra pozostaje ściśle < 1 . Dzięki temu każde naruszenie ułożonego kubika (wartość całkowita) zawsze dominuje nad błędem klastra. Wzdłuż trajektorii GENES_COUNT ruchów śledzimy najlepszy krok i zapamiętujemy jego długość jako MovesCount.

Ten sam wzór jest zaimplementowany po stronie CPU jako `TRubikCube.EvaluateGpu()` — liczy fitness aktualnej kostki (osobnik z zerem ruchów), aby wyznaczyć **baseline** porównywalny z wynikiem z GPU.

6. Selekcja, krzyżowanie, sortowanie

6.1 Sortowanie bitoniczne

Populacja jest sortowana rosnąco względem fitnessu **siecią bitoniczną** (POPULATION_COUNT jest potęgą 2). Host steruje etapami i przejściami przez uniformy `u_Stage` / `u_PassModStage`; po sortowaniu najlepszy osobnik jest na indeksie 0.

6.2 Selekcja i krzyżowanie (SelCrossover)

Każdy wątek tworzy jedno dziecko:

- **Selekcja rankingowa**: dwaj różni rodzice losowani z górnych WINNERS_RATIO% już posortowanej populacji.
- **Krzyżowanie odwrotno-komplementarne** (wiernie odwzorowanie operatora CPU): pierwsza część genomu to split-mix genów obojga rodziców, a druga połowa to odwrócona sekwencja z zanegowanymi kątami (`getRevCode`). Wykorzystuje to strukturę odwracalności, na której opiera się fitness.
- **Mutacja** z prawdopodobieństwem MUTATION_RATIO%: podmiana jednego genu w pierwszej połowie na losowy `FreeMove`, przed odbudową komplementu.

Dzieci trafiają do bufora `NewPopulation` (binding 1), który po zamianie ping-pong staje się populacją rodziców kolejnej generacji.

7. Integracja z pętlą klastrów (host)

Kostka rozwiązywana jest **klaster po klastrze**. `NextCluster` wyznacza kolejny aktywny klaster (kubiki o najmniejszym indeksie klastra, które nie są jeszcze ułożone). Dla nowego klastra `baseline` liczony jest metryką GPU (`EvaluateGpu`) i zapisywany w `RubikCube.Score` — to jednocześnie próg przerwania GA (gdy `best.Fitness < Score`) i akceptacji znalezionej sekwencji ruchów. Ruchy najlepszego osobnika są aplikowane na CPU (`Turn`), a gdy `Score` spadnie do 0, klaster jest ułożony i przechodzimy do następnego.

7.1 Jedna reprezentacja stanu i pakowanie

Stan kostki na CPU trzyma **wyłącznie** `Transform` (macierz orientacji + pozycja per kubik). Wcześniejsza równoległa reprezentacja SOA (tablice M/P pod SIMD) została usunięta jako zbędne dublowanie — GPU i tak jest szybsze. `PackCubies` pakuje macierz do formatu shadera wprost z `Transform.M` (obroty 90° dają dokładne 0/±skala, więc pakowanie jest jednoznaczne).

7.2 Cykl życia buforów

Bufory tworzone są **raz** i reużywane w kolejnych przebiegach: `Ssbo.Update` wywołuje `glBufferData`, które realokuje magazyn przy zmianie rozmiaru, zachowując to samo id i binding. Eliminuje to churn buforów (częste tworzenie/usuwanie), będący typowym źródłem niestabilności sterownika.

8. Uwagi implementacyjne i pułapki

- **Reinterpretacja fitnessu:** GPU zapisuje `floatBitsToUint(fitness)`, host odczytuje bitowo przez `BitConverter.UInt32BitsToSingle`. Sortowanie bitoniczne działa na wzorcu bitowym (poprawne dla $\text{fitnessu} \geq 0$).
- **Liczności klastra jako uniformy:** `countSolved / countActive` pochodzą z hosta, a nie z `.length()` bufora — pierwszy klaster ma pusty bufor `SolvedCubies`, a `.length()` na SSBO rozmiaru 0 jest niepewne między sterownikami.
- **Wyrównanie std140:** tablica `ivec2 Planes[]` w UBO ma każdy element wyrównany do 16 bajtów (`vec4`), więc host uploaduje po 4 inty na element (`x, y + padding`).
- **Obejście błędu sterownika:** w `getStartCoordinate` ogólny wzór musi leżeć w bloku `else`. Wariant z wczesnym `return` tuż po `if (SIZE==2)` był miscompilowany przez sterownik (funkcja `inline`'owana do gorącej pętli), co powodowało `Access Violation` zależny od danych.

Podsumowanie: potok GPU-GA oddziela współdzielone obliczenia (klasa `Gpu`) od renderowania per-okno (`TGLContext`). Reprezentacja kubika jako spakowana macierz w pojedynczym `uint` pozwala symulować ruchy w rejestrach, a metryka `L1` z dominującym znakiem daje gładki, monotoniczny krajobraz fitnessu dla algorytmu genetycznego.